

CLOUD-BASED DISASTER RECOVERY SYSTEM

**AWS Environment Design and
Implementation**

Name:

Archie Chadha

TABLE OF CONTENT

1. Introduction.....	2
I. General short description of the AWS Cloud.....	2
II. How does the elasticity of Amazon EC2 impact on computation in a cloud architecture designed to handle variable workloads?	3
III. How can the automated backup and failover features of Amazon RDS contribute to a disaster recovery plan, and what are the potential pros and cons when relying solely on these features?	4
IV. What configurations must be considered regarding cold starts and function concurrency in AWS Lambda's serverless architecture?	6
V. Problem statement:.....	7
2. Screenshots	9
3. Scenario Overview.....	11
4. List of References	14

1. Introduction

I. General short description of the AWS Cloud.

A complete platform for cloud computing, **Amazon Web Services (AWS)** provides a vast range of on-demand services, such as processing power, storage, databases, networking, analytics, and more. Businesses of all sizes may create complex applications with greater flexibility, scalability, and dependability by utilizing AWS's scalable, adaptable, and affordable solutions. (*What-is-aws*, n.d.)

Elastic Compute Cloud (EC2), which enables users to rent virtual machines (instances) on demand, is one of AWS's primary services. This enables enterprises to install and maintain apps at scale and does away with the requirement to purchase physical hardware.

Cloud Computing

By dispersing applications over several regions and availability zones, AWS additionally guarantees high availability and disaster recovery. Through failover techniques, this feature makes sure that programs continue to function even in the case of a regional failure.

Because AWS uses a pay-as-you-go business model, customers don't have to make upfront hardware purchases; instead, they just pay for the services they utilise. Because of its affordable price and wide range of services (which include analytics, machine learning, storage, and computing), AWS is the go-to option for businesses of all sizes, including startups and governmental agencies.

II. How does the elasticity of Amazon EC2 impact on computation in a cloud architecture designed to handle variable workloads?

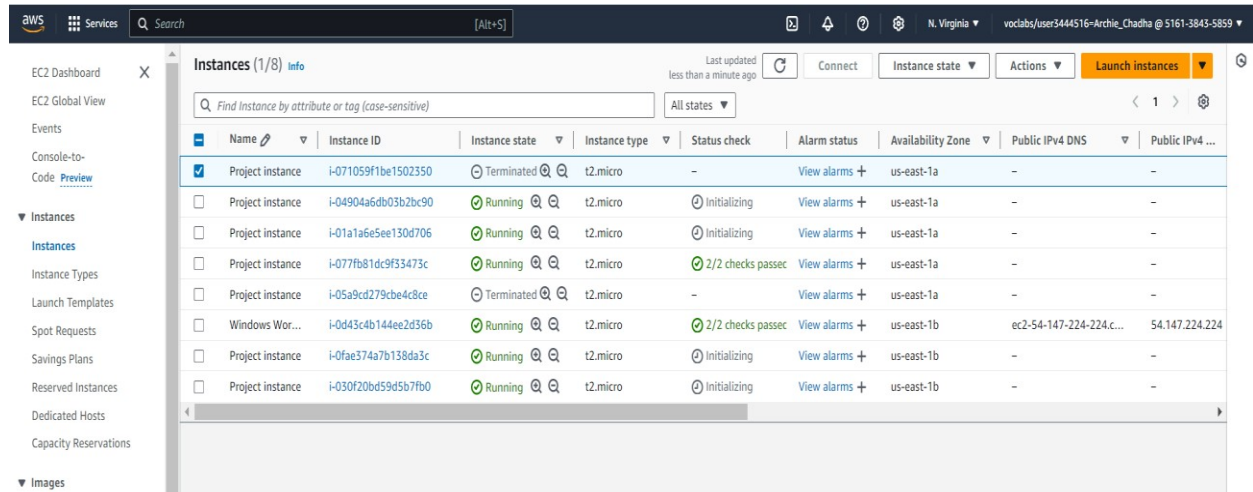
One of the main characteristics of cloud computing is elasticity, which is best represented by Amazon EC2, which enables apps to automatically scale resources in response to demand. For current web applications, where traffic might change rapidly and manual scaling would be wasteful and expensive, this flexibility is essential (*Instance Auto Scaling - Amazon EC2 Auto Scaling - AWS*, n.d.).

I set up an Auto Scaling Group in the AWS lab configuration that dynamically modifies the quantity of EC2 instances in response to CloudWatch alarms related to CPU utilization. To maintain optimal performance, the Auto Scaling Group automatically adds new instances when the CPU consumption surpasses a predetermined threshold (e.g., 60%). Similarly, the group minimizes expenses by reducing the number of instances when CPU utilization falls below a predetermined threshold (e.g., 45%). This is done by removing superfluous resource usage.

Because it guarantees that companies are only paying for the resources they require, EC2's elasticity has an impact on cloud architecture. When managing fluctuating workloads, like a web application that experiences spikes in user traffic, this is quite helpful. In order to maintain a seamless user experience, more EC2 instances are automatically deployed during periods of high demand. On the other hand, instances end when traffic is light, which lowers running expenses (*Instance Auto Scaling - Amazon EC2 Auto Scaling - AWS*, n.d.).

Screenshot 1: Showing Results

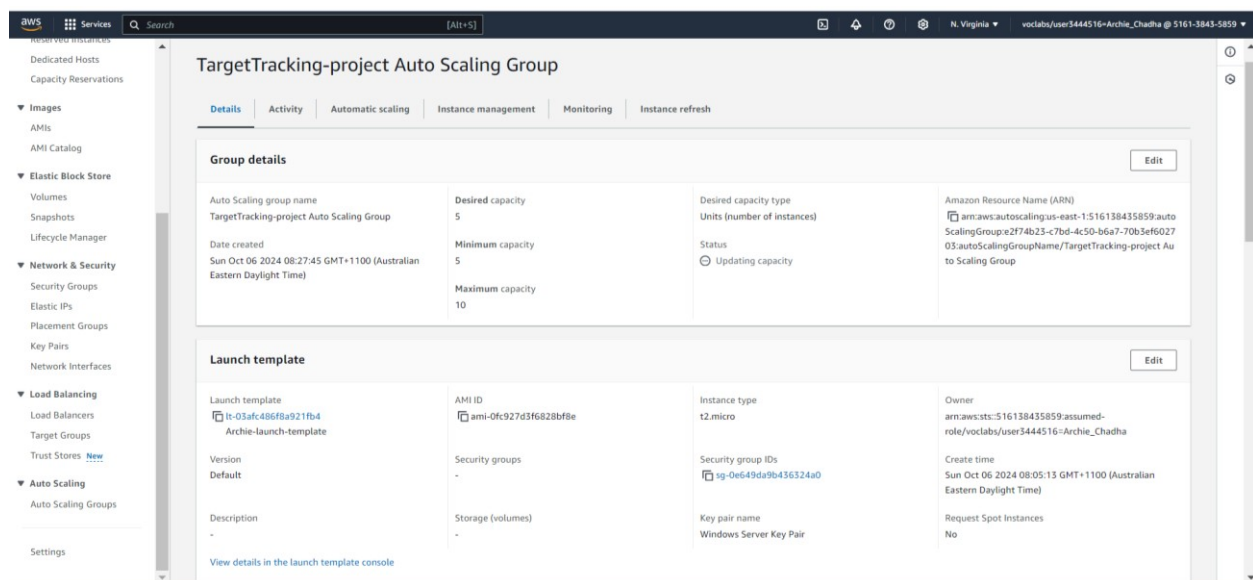
Cloud Computing



This screenshot shows the AWS Management Console's 'Instances' page. The left sidebar contains navigation links for EC2 Dashboard, Global View, Events, Console-to-Code, and various EC2 services like Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, and Capacity Reservations. The main content area displays a table of EC2 instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, Public IPv4 DNS, and Public IPv4 address. There are 8 instances listed, with one in a 'Terminated' state and seven in a 'Running' state. The 'Running' instances are of type 't2.micro' and are located in 'us-east-1a' or 'us-east-1b' availability zones. The status checks for the running instances show '2/2 checks passed'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
Project instance	i-071059f1be1502350	Terminated	t2.micro	-	View alarms +	us-east-1a	-	-
Project instance	i-04904a6db03b2bc90	Running	t2.micro	Initializing	View alarms +	us-east-1a	-	-
Project instance	i-01a1a6e5ee130d706	Running	t2.micro	Initializing	View alarms +	us-east-1a	-	-
Project instance	i-077fb81dc9f33473c	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	-
Project instance	i-05a9cd279cbe4c8ce	Terminated	t2.micro	-	View alarms +	us-east-1a	-	-
Windows Wor...	i-0d43c4b144ee2d36b	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-54-147-224-224.c...	54.147.224.224
Project instance	i-0fae574b7b138da3c	Running	t2.micro	Initializing	View alarms +	us-east-1b	-	-
Project instance	i-030f20bd59d5b7fb0	Running	t2.micro	Initializing	View alarms +	us-east-1b	-	-

Screenshot 2: Auto Scaling Group



This screenshot shows the 'TargetTracking-project Auto Scaling Group' page in the AWS Management Console. The left sidebar is the same as the previous screenshot. The main content area has tabs for Details, Activity, Automatic scaling, Instance management, Monitoring, and Instance refresh. The 'Details' tab is selected, showing 'Group details' and 'Launch template' sections. The 'Group details' section includes fields for Auto Scaling group name, Date created, Desired capacity, Minimum capacity, Maximum capacity, Desired capacity type, Status, and Amazon Resource Name (ARN). The 'Launch template' section includes fields for Launch template, AMI ID, Instance type, Security groups, Security group IDs, Key pair name, Owner, Create time, and Request Spot Instances.

Group details			
Auto Scaling group name	Desired capacity	Desired capacity type	Amazon Resource Name (ARN)
TargetTracking-project Auto Scaling Group	5	Units (number of instances)	arn:aws:autoscaling:us-east-1:516138435859:auto-scaling-group:274b25-c7bd-4c50-b6a7-70b3ef602703:autoScalingGroupName/TargetTracking-project Au
Date created	Minimum capacity	Status	
Sun Oct 06 2024 08:27:45 GMT+1100 (Australian Eastern Daylight Time)	5	Updating capacity	
	Maximum capacity		
	10		

Launch template			
Launch template	AMI ID	Instance type	Owner
lt-03af486f8a921fb4	ami-0fc927d3f6828bffe	t2.micro	arn:aws:sts:516138435859:assumed-role/voclabs/user3444516-Archie_Chadha
Archie-launch-template			
Version	Security groups	Security group IDs	Create time
Default	-	sg-0e649da9b436324a0	Sun Oct 06 2024 08:05:13 GMT+1100 (Australian Eastern Daylight Time)
Description	Storage (volumes)	Key pair name	Request Spot Instances
-	-	Windows Server Key Pair	No

III. How can the automated backup and failover features of Amazon RDS contribute to a disaster recovery plan, and what are the potential pros and cons when relying solely on these features?

Disaster recovery is essential in cloud systems to reduce downtime and data loss in the event of a system breakdown. An organization's disaster recovery plan is greatly aided by the built-in

automated backup and failover procedures that Amazon Relational Database Service offers (*Cloud Relational Database – Amazon RDS Features – Amazon Web Services*, n.d.).

Point-in-time recovery is made possible via automated backups in RDS, which guarantees that users can restore their databases to any point within the backup retention period in the event of a failure. When handling inadvertent deletions, corrupted data, or device malfunctions, this capability is priceless.

In the event of an instance loss, RDS's failover feature instantly switches to a backup replica in a different availability zone to guarantee high availability. By doing this, downtime is reduced, and applications continue to operate without user involvement (*Cloud Relational Database – Amazon RDS Features – Amazon Web Services*, n.d.). On the other hand, using automatic backup and failover provides advantages and disadvantages:

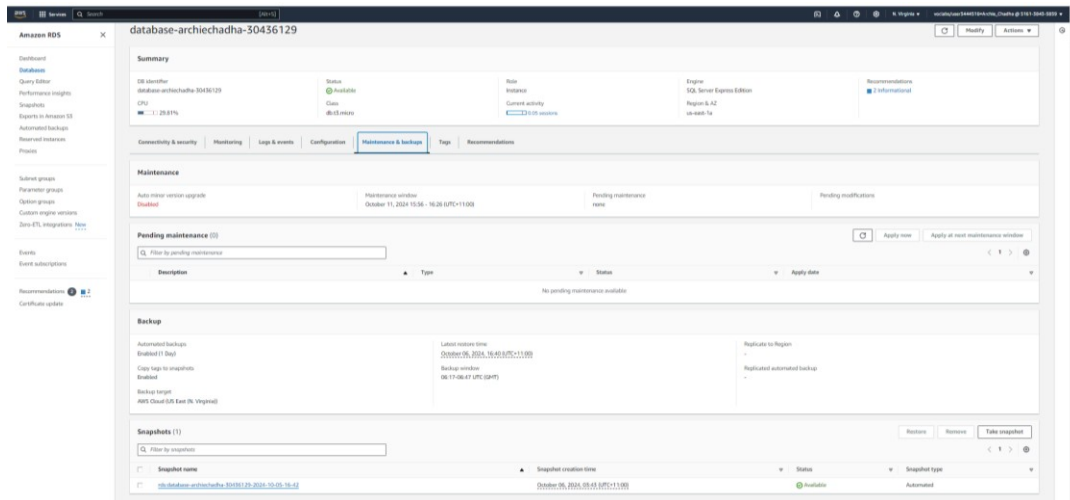
Pros:

- Point-in-time recovery: Enables the restoration of a database to any point during the retention period.
- Automated backups: RDS may automatically produce backups of your database, lowering the chance of data loss.
- Multi-AZ deployments: Offer automatic failover to a standby instance in the event of an infrastructure breakdown and high availability.

Cons:

- Limited control: Customizing backup and recovery procedures may be restricted if one just uses RDS features.
- Potential expenses: Deploying multiple AZs and keeping backups for longer periods of time may raise costs.
- Recovery time: Recovery may take longer than with custom solutions depending on the size of the database.

Screenshot 3: Showing Automated RDS backup



IV. What configurations must be considered regarding cold starts and function concurrency in AWS Lambda's serverless architecture?

With serverless computing, which is made possible by AWS Lambda, code can be run in response to events without requiring server provisioning or management. Cold starts, on the other hand, can happen to Lambda functions when a fresh instance of the function is initialized following a period of inactivity. This results in longer response times, particularly for the initial request, which adds latency (*Serverless Land*, 2023).

In settings where functions are triggered rarely, cold starts are typically more evident. To reduce cold start times, AWS offers provisioned concurrency, which maintains a preset number of instances warm and available to handle requests. However, there is an additional fee for this service, and function concurrency must be carefully handled.

The quantity of concurrent executions that a certain function can support in Lambda is known as concurrency. To prevent backend services from being overloaded by heavy traffic events, AWS gives developers the option to set concurrency limitations. This is critical for scaling programs effectively while retaining performance (*Serverless Land*, 2023).

AWS Lambda configuration for best performance requires considering several criteria related to function concurrency and cold starts:

- **Provisioned Concurrency:** This function lowers cold start latency by enabling the prewarming of a predetermined number of execution environments (*What-is-aws*, n.d.).
- **Memory Allocation:** Increasing the memory allotted will shorten the time needed for a cold start and enhance function performance.
- **Code Optimization:** Cold start times can be shortened by reducing dependencies and improving initialization code.

- Reusing Execution Environments: Lambda can be used to minimize cold starts by keeping execution environments warm for a while after invocation (*Serverless Land*, 2023).
- Reserved Concurrency: By imposing a reserved concurrency restriction, important functions are guaranteed a certain number of concurrent executions.

V. Problem statement:

A business runs its web application on Amazon EC2 instances and intends to employ an Auto Scaling Group to scale its infrastructure. As of right now, the business is operating 28 EC2 instances, which are determined by adding the numbers in the student ID (30436129). The Auto Scaling Group automatically creates more instances to handle the load as traffic climbs by 50% and then reduces those instances when demand decreases. The task at hand is to compute:

- The total instance-hours utilized during the peak traffic period.
- The entire amount spent during the peak.

The company currently has N instances running where N is the sum of the digits of my student ID (30436129). The sum of the digits is:

$$3 + 0 + 4 + 3 + 6 + 1 + 2 + 9 = 28$$

- Let the base number of instances be $N = 28$.
- Traffic increases by 50%, prompting the Auto Scaling Group to launch M additional instances.
- $M = N \times 50\% = 28 \times 0.5 = 14$ instances.
- The spike lasts 5 hours.
- After the spike subsides, the Auto Scaling Group scales back down to the original $N = 28$ instances.

1. Total Instance-Hours Used During the Spike:

- Before the spike: 28 instances are running.
- During the spike: 28 original instances + 14 additional instances = 42 instances are running.
- Spike duration: 5 hours.

Now, calculate the instance-hours:

- For the original 28 instances:

$$28 \text{ instances} \times 5 \text{ hours} = 140 \text{ instance-hours.}$$

- For the 14 additional instances:

$$14 \text{ instances} \times 5 \text{ hours} = 70 \text{ instance-hours.}$$

Cloud Computing

- Total instance-hours: 140
+ 70 = 210 instance-hours.

2. Total Cost Incurred During the Spike:

We are using t3.medium EC2 instances in the US East (N. Virginia) region, where the cost per instance-hour is approximately \$0.0416.

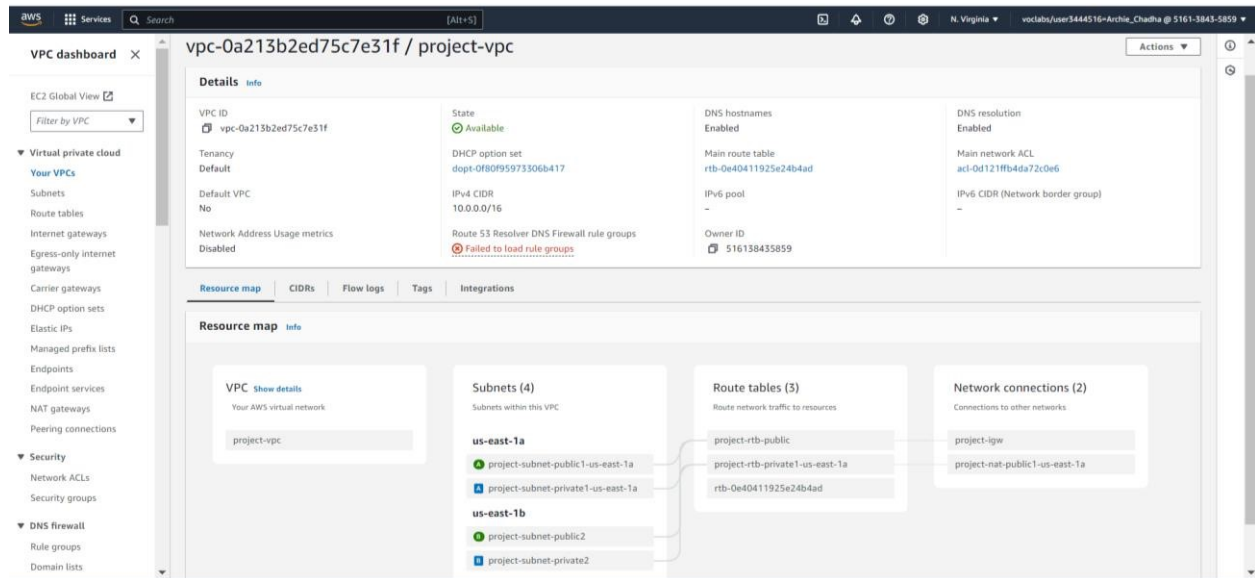
- Total cost:
 $210 \text{ instance-hours} \times \$0.0416 = \$8.74.$

Summary:

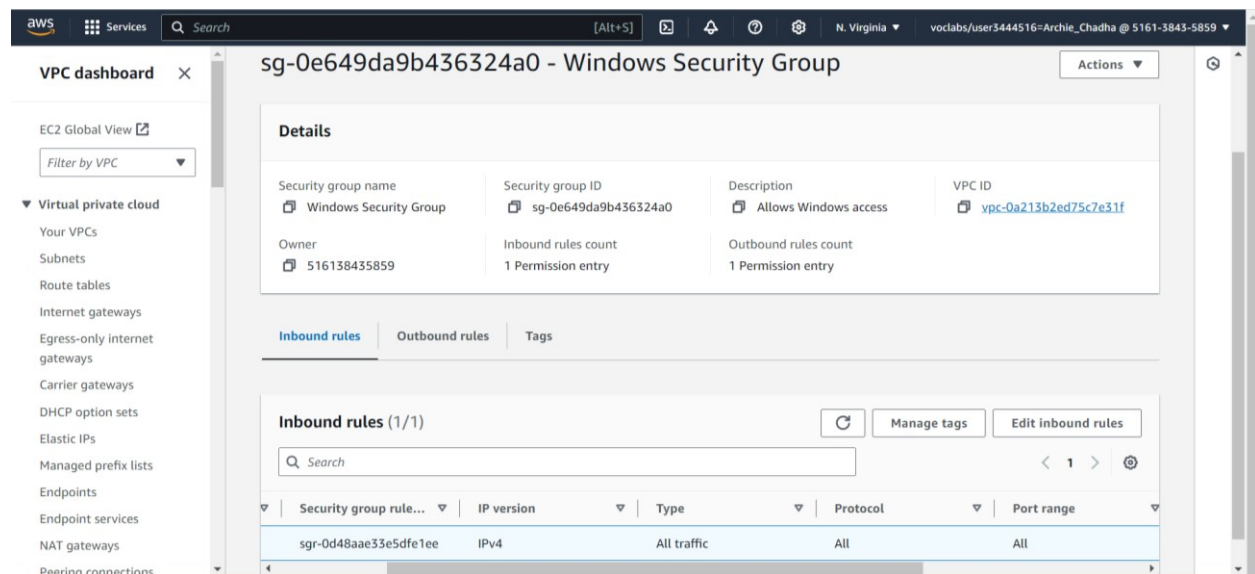
- Total instance-hours used during the spike: 210 instance-hours.
- Total cost incurred during the spike: \$8.74

2. Screenshots

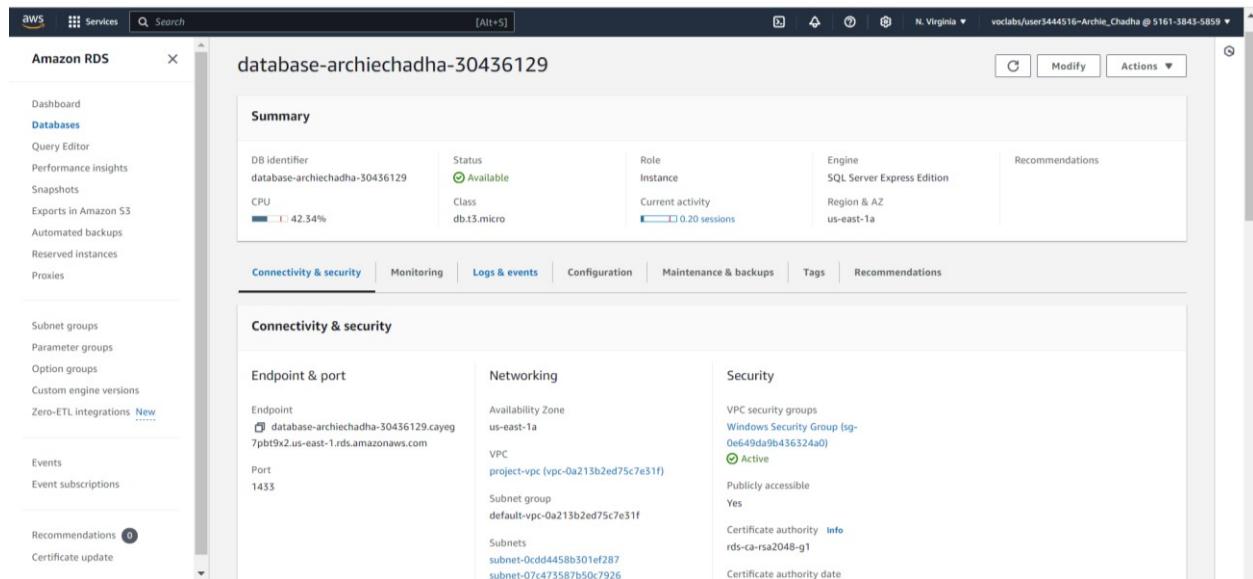
Screenshot 1: VPC Resource map.



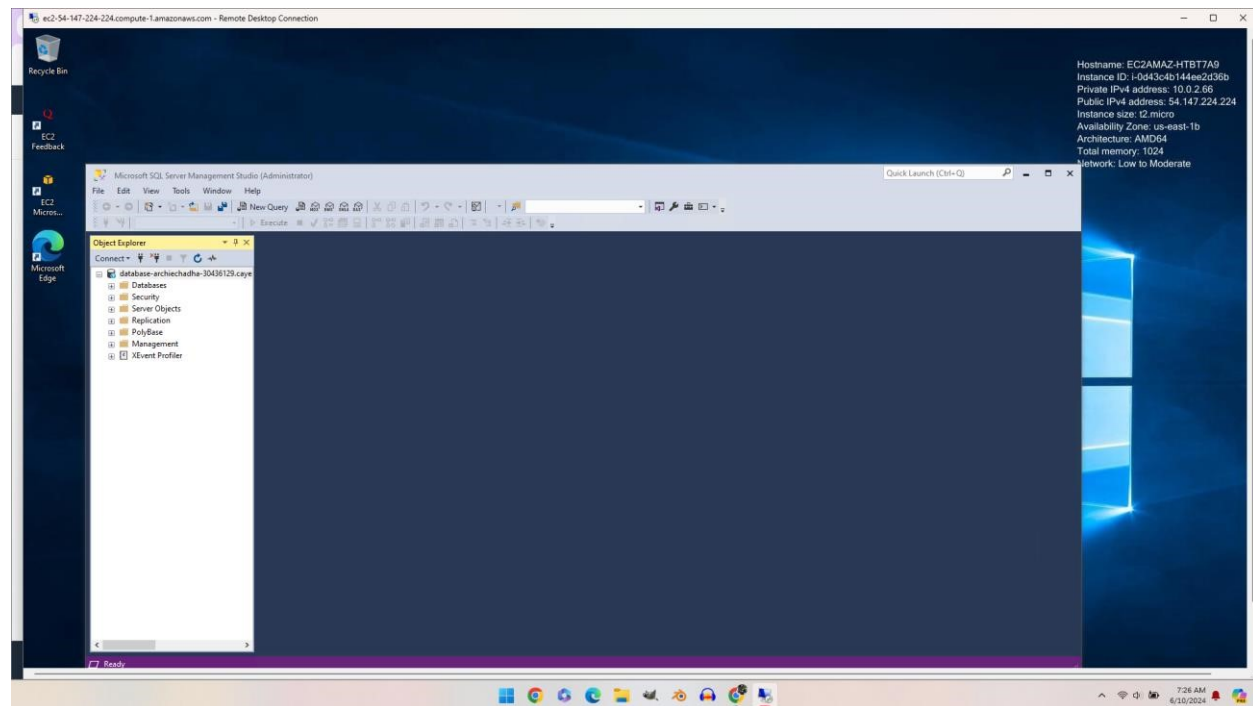
Screenshot 2: Security group.



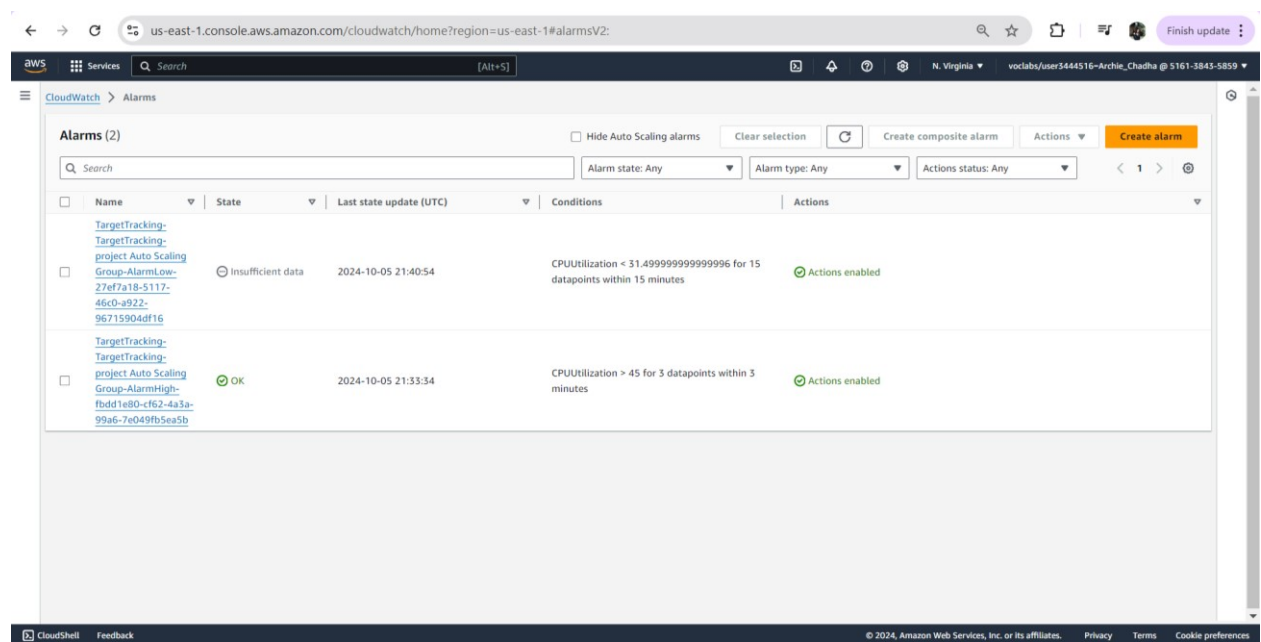
Screenshot 3: RDS



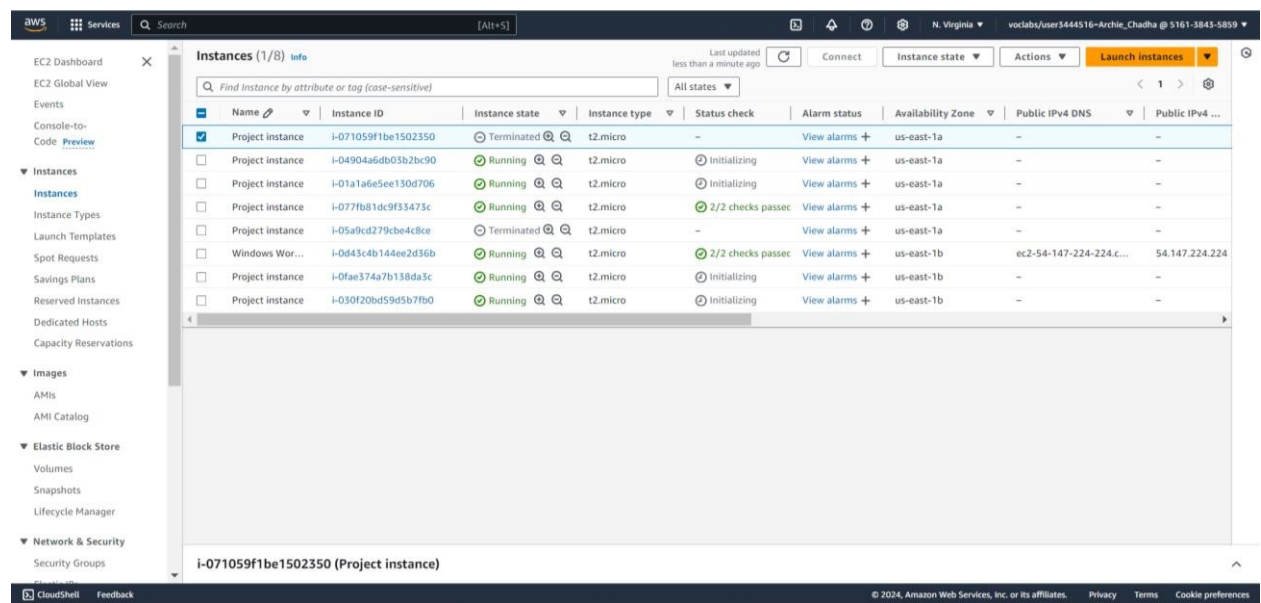
Screenshot 4: SQL Server Management Studio



Screenshot 5: CloudWatch AlarmLow and AlarmHigh



Screenshot 6: New Instance(s) created by Auto Scaling



3. Scenario Overview

3.1 Explanation of the Scenario

Through the implementation of a Virtual Private Cloud (VPC) spanning two Availability Zones (A and B), I have created a reliable and highly available AWS environment for this project. There are two public and two private subnets in the architecture. While the private subnets house vital components, like an Amazon RDS DB instance, they also ensure high availability and security. The public subnets are used for resources that are visible to the outside world. I set up an EC2 instance in the public subnet, running a Windows server with SQL Server Management Studio (SSMS) installed for database management, in order to communicate with the RDS instance.

Virtual Private Cloud (VPC)

I set up a VPC with a CIDR block of 10.0.0.0/16, so the environment will have enough IP addresses for expansion in the future and more subnets if needed. In order to ensure that resources are dispersed throughout various zones for improved availability, the VPC is built to host both public and private subnets in distinct Availability Zones (AZs). **Subnets**

- **Public Subnet 1 (10.0.0.0/24):** The NAT gateway is installed in Availability Zone A, where Public Subnet 1 is located. The NAT gateway can enable outbound internet access for instances located in private subnets thanks to this subnet. Additionally, it houses an EC2 Windows Workstation, which serves as the primary resource access interface.
- **Private Subnet 1 (10.0.1.0/24)** in Availability Zone A, which houses the main Amazon RDS DB instance. To protect the database, only the EC2 instance in Public Subnet 1 is permitted access to this subnet, which is segregated from direct internet access.
- **Public Subnet 2 (10.0.2.0/24):** This subnet can be used as a backup location for resources that require internet connection. It is installed in Availability Zone B. It guarantees redundancy for future expansion even though it's not in use for this scenario right now.
- **Private Subnet 2 (10.0.3.0/24):** As part of the high-availability (HA) configuration, Private Subnet 2, which is situated in Availability Zone B, is home to the secondary Amazon RDS Database instance. This configuration enables failover protection in case the primary database becomes unavailable.

Security Group

To control incoming and outgoing traffic, I established and set up security groups. Two incoming rules are configured in the EC2 instance's security group:

- **All Traffic:** During the configuration stage, this feature permits unlimited access to the instance for testing reasons.
- **MSSQL Traffic (Port 1433):** This port is solely open for communication pertaining to databases and is used to limit traffic to SQL Server Management Studio.

To provide safe and restricted access, the security group for the RDS instance is set up to only allow inbound traffic from the Windows server inside the VPC. To enable smooth communication

while preserving the security perimeter, the configuration makes sure that the RDS and EC2 instance are in the same security group.

AWS RDS Database Instance

I set up an Amazon RDS database instance in each of the two Availability Zones' private subnets (A and B). Because the RDS DB instance is replicated among various zones, high availability is guaranteed, and failover capabilities are provided in case one zone experiences outage. To ensure that the database is always accessible, the backup instance is located in Private Subnet 2 (AZ B) while the primary instance is deployed in Private Subnet 1 (AZ A).

Applications that depend on the RDS instance will benefit from redundancy and reliability thanks to its multi-AZ configuration, which offers high availability and automatic failover. To ensure secure database activities, the database instance can only be accessed through the Windows EC2 instance via SQL Server Management Studio (SSMS).

Elastic Load Balancing (ELB) and Auto Scaling

The EC2 instances are connected to an **elastic load balancer (ELB)**, which disperses incoming traffic among them. This lessens the chance that the traffic load will exceed any one instance. Through the use of several EC2 instances, traffic balancing, or ELB, applications are kept responsive and available even during periods of high usage.

Auto Scaling Group: Depending on traffic need, I set up an Auto Scaling Group to dynamically change the number of instances that are running. CloudWatch alerts, which track important metrics like CPU utilization, are linked to this group. The Auto Scaling Group starts more instances when the CPU utilization goes beyond 60% to accommodate the extra traffic. It saves money and resources by terminating unnecessary instances when the CPU utilization falls below 45%.

For applications with erratic or fluctuating traffic, like streaming services or e-commerce platforms, this combination of ELB with Auto Scaling is perfect. The system can react rapidly to variations in demand while minimizing operating expenses during times of low traffic by autonomously scaling the infrastructure.

CloudWatch Alarms and Scaling Operations

In order to initiate scaling steps based on CPU utilization, I used CloudWatch alerts. This is how the procedure operates:

- The Auto Scaling Group launches more EC2 instances to spread the workload when a CloudWatch alarm exceeds 60% CPU utilization.
- On the other hand, another warning indicates that instances should be terminated to save resources and money when CPU utilization drops below 45%.

By guaranteeing that the infrastructure automatically adjusts to traffic levels without human involvement, these alerts aid in the optimization of resource consumption. The environment is

more economical and efficient when these alerts are configured, scaling down during periods of low demand and scaling up during off-peak period.

3.2 Technical Applicability

This configuration is ideal for companies that need web applications that are robust and scalable. The application can withstand traffic spikes with minimal downtime because to the combination of load balancing, highly available RDS, and auto-scaling EC2 instances. Moreover, developers may handle database activities more effectively by using SQL Server Management Studio to manage the database, which enhances operational efficiency.

3.3 Business Applicability

The flexibility and Auto Scaling features of AWS offer a financially viable option for businesses dealing with varying traffic volumes. Through automatic additions or deletions of instances in response to current demand, businesses can minimise infrastructure costs and steer clear of overinvesting in off-peak times. This methodology additionally guarantees the application's availability and responsiveness through abrupt spikes in traffic, thereby augmenting the user experience in its whole.

Conclusion

This AWS cloud example shows how Auto Scaling, Elastic Load Balancing, and RDS high availability work together to create a potent combination. Businesses may maintain the scalability, resilience, and affordability of their infrastructure by utilising these services. Robust disaster recovery and optimal performance are provided by the automated backups and failover features of RDS, in conjunction with the dynamic scalability enabled by EC2 instances.

4. List of References

- Amazon Web Services. (2024). What is AWS? <https://aws.amazon.com/what-is-aws/>
- Amazon Web Services. (2024). Amazon EC2 Auto Scaling. <https://aws.amazon.com/ec2/autoscaling/>

- Amazon Web Services. (2024). Amazon RDS Features.
<https://aws.amazon.com/rds/features/>
- Amazon Web Services. (2024). AWS Lambda - Provisioned Concurrency.
<https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>
- Serverless Land. (2023). Best Practices for Lambda Performance Optimization.
<https://serverlessland.com/best-practices/lambda-optimization>
- Amazon Web Services. (2024). Auto Scaling Groups.
<https://docs.aws.amazon.com/autoscaling/ec2/userguide/AutoScalingGroup.html>
- Gartner. (2023). Cloud Computing Benefits.
<https://www.gartner.com/en/informationtechnology/insights/cloud-strategy>